

WAYPOINT

Real-Time Fleet & Delivery Tracking Platform

PRODUCT & SYSTEM REQUIREMENTS DOCUMENT (PRD/SRD)

Document Version: 1.0

Prepared for: Raynald — Backend Engineering Portfolio Project

Date: July 2026

1. Product Overview

1.1 What Is Waypoint?

Waypoint is a backend-only, real-time fleet and delivery tracking system. It solves the same core technical problem that powers apps like ride-hailing and food delivery platforms: showing a customer exactly where their driver or shipment is, live, on a map, without delay — while keeping every user's data strictly isolated from every other user.

This is a portfolio and learning project. It does not have a polished driver/customer mobile app; the goal is to build and prove out the backend system that would sit behind such an app. The emphasis throughout is on the mechanisms that senior engineers and system design interviewers care about: real-time bidirectional communication, high-frequency data ingestion, targeted broadcast (not blind broadcast), geospatial computation, and state consistency under concurrent load.

1.2 Problem Statement

Real-time location tracking is deceptively hard to build correctly. Naive implementations poll the server every few seconds, which wastes server resources, adds latency, and doesn't scale past a small number of concurrent users. Systems that don't scope data properly risk leaking one customer's driver location to another customer. Systems that treat location data like ordinary relational data end up with bloated tables and slow queries, because location updates arrive far more frequently than they need to be permanently stored.

Waypoint exists to solve these problems directly and demonstrate the correct architectural patterns: persistent bidirectional connections (WebSocket), topic-scoped delivery (publish-subscribe), a fast-changing in-memory data layer (Redis) separate from the durable system of record (PostgreSQL), and clear authorization boundaries enforced at the data layer, not just the UI.

1.3 Goals of This Project

- Build a working, demonstrable real-time tracking backend end-to-end.
- Practice and prove system design skills that map directly to common big-tech interview questions (e.g., "design Uber's location tracking," "design a live delivery tracker").
- Extend previously learned skills (Spring Boot, JPA, Redis, JWT/RBAC, Docker, CI/CD) into a new problem domain (real-time systems) rather than repeating CRUD-only patterns.
- Produce a project that is easy to explain to both technical interviewers and non-technical people, with a clear before/after value story.
- Ship something deployed, working, and demoable — not just code that runs locally.

1.4 Non-Goals (Explicitly Out of Scope)

To keep this achievable within one month, the following are intentionally excluded. These are good "future work" talking points in an interview, but are not part of the MVP:

- Real turn-by-turn routing via a mapping provider (Google Maps / Mapbox routing API) — ETA is computed with straight-line distance and average speed, not real road routing.
- Payments, invoicing, or any monetary transaction processing.

- A production-grade mobile app UI (driver or customer). A minimal web-based map viewer is enough to visually demonstrate tracking; drivers are simulated via a script, not real GPS hardware.
- Multi-region / multi-datacenter deployment. Single-region deployment is sufficient.
- Machine-learning-based ETA prediction or driver-matching optimization.

2. Users & Personas

2.1 User Roles

Role	Description	Primary Needs
Customer	Places an order/delivery request and wants to track its progress.	See live driver location, order status, ETA.
Driver	Fulfills deliveries; moves physically and reports location.	Receive assigned orders, send location updates, update order status.
Dispatcher / Admin	Oversees fleet operations.	See all active orders, all driver statuses, system health, override assignments if needed.

2.2 Core User Stories

1. As a Customer, I want to place an order and be automatically matched with the nearest available driver, so that I don't have to manually pick one.
2. As a Customer, I want to see my assigned driver's location update live on a map, so that I know when to expect my delivery.
3. As a Customer, I want to see an estimated time of arrival, so that I can plan around it.
4. As a Customer, I must not be able to see any other customer's order or driver location, even by guessing an ID.
5. As a Driver, I want to toggle my availability on/off, so that I only receive orders when I'm actually working.
6. As a Driver, I want my app to continuously report my location while I have an active order, so the customer can track me.
7. As a Driver, I want to update the delivery status (picked up, on the way, delivered), so the system and customer stay informed.
8. As a Dispatcher, I want a live overview of all active orders and driver statuses, so I can spot problems (e.g., a stuck delivery) quickly.
9. As a Dispatcher, I want to view the full historical route of any past order, so I can investigate complaints or disputes.
10. As the System, I must reassign or flag an order if a driver disconnects or goes offline mid-delivery, so no order silently stalls.

3. Feature Specification

Features are grouped into MVP (must-have to be considered "done") and Stretch (build only if time remains after MVP is solid and deployed).

3.1 MVP Features

F1 — Authentication & Role-Based Access

Users register/login as Customer, Driver, or Dispatcher. JWT-based authentication, same pattern as prior projects. Every protected endpoint and WebSocket subscription is authorized based on role and ownership, not role alone.

- Sign up / login endpoints for all three roles.
- JWT issued on login, required on all subsequent requests and WebSocket handshakes.
- Role guard: Dispatcher-only endpoints reject Customer/Driver tokens.
- Ownership guard: a Customer can only fetch/subscribe to their own orders; a Driver can only update their own assigned order.

F2 — Order Lifecycle Management

An Order is the central entity representing one delivery request. It moves through a fixed set of stages, validated the same way stage transitions were validated in the previous Hireloop project.

- Stages: CREATED → ASSIGNED → PICKED_UP → ON_THE_WAY → DELIVERED, with CANCELLED reachable from CREATED or ASSIGNED.
- Every transition is validated against an allowed-transitions map (no skipping stages, no illegal reversals).
- Every transition is timestamped and stored for history/audit.

F3 — Nearest-Driver Assignment

When a Customer creates an order, the system finds the closest available Driver and assigns them automatically.

- Query all drivers currently marked AVAILABLE and ONLINE.
- Compute distance from each candidate driver's last known location to the order's pickup point using the Haversine formula.
- Assign the nearest one; mark them BUSY; move order to ASSIGNED.
- If no driver is available, order stays in CREATED with a clear "no driver available" state (not a silent failure).

F4 — Real-Time Location Ingestion (Driver → Server)

While a Driver has an active order, their client sends location updates at a fixed interval over a persistent WebSocket connection.

- Transport: WebSocket using STOMP protocol.
- Update frequency: every 3–5 seconds (configurable) while an order is active.
- Each update is validated (plausible latitude/longitude range, belongs to a driver with an active order).

F5 — Real-Time Location Broadcast (Server → Customer)

The moment a location update is received, it is pushed to exactly the customer who owns that order — no one else.

- Each order gets its own topic/channel (e.g. a per-order destination).
- Only the owning customer's session may subscribe to that specific order's topic; this is enforced at subscribe-time, not just at the UI.
- Latest location is also cached in Redis for instant retrieval by clients that just opened the tracking view (no need to wait for the next update).

F6 — ETA Calculation (Simplified)

A lightweight, explainable ETA — not real routing — computed from straight-line distance and a configurable average speed assumption.

- Recomputed on every location update.
- Returned alongside each location broadcast so the customer's UI can show "Arriving in ~8 minutes."

F7 — Location History & Trip Replay

While live location is ephemeral (Redis), every update is also durably appended to a history log so a completed trip's full route can be reconstructed later.

- History write happens asynchronously so it never blocks or slows down the live broadcast path.
- A dispatcher (or the customer, for their own completed order) can fetch the full route of a finished order.

F8 — Driver Availability & Presence

Drivers explicitly go online/offline and available/busy. The system also detects abrupt disconnects (app closed, network dropped) rather than assuming a driver is still online forever.

- Explicit status toggle endpoint/WebSocket event.
- Heartbeat or WebSocket disconnect handling: if a driver's connection drops mid-order, the order is flagged for dispatcher attention rather than left silently "in progress" forever.

F9 — Dispatcher Operations Dashboard (API)

A read-heavy set of endpoints giving an operational overview. This endpoint is cached, mirroring the caching pattern used for the analytics dashboard in the previous project.

- Count of active orders by stage.
- Count of online/available/busy drivers.
- List of orders flagged for attention (stalled, disconnected driver, no driver available).

F10 — Rate Limiting on Public-Facing Endpoints

The order-creation endpoint and login endpoint are protected against abuse using the same Redis-counter rate limiting pattern learned previously.

- Per-IP and per-account limits on order creation.
- Proper 429 responses with a retry-after hint.

3.2 Stretch Features (Only If Time Remains)

- Driver rating system after order completion.
- Multi-stop orders (one driver, several drop-off points in sequence).
- Basic map-based web viewer (a simple HTML/JS page consuming the WebSocket, purely for demo purposes).
- Push-style in-app notifications (e.g., "your driver is 2 minutes away") as a dedicated notification service.
- Admin manual reassignment UI/endpoint for stuck orders.

4. System Architecture

4.1 High-Level Architecture (Whiteboard-Reproducible)

This is described as a set of components and connections rather than fragile ASCII art, so it can be redrawn confidently on an actual whiteboard in an interview. Practice sketching this from the breakdown below until it is second nature.

Clients

- Customer Client — connects over HTTPS (REST) to create orders, and over WebSocket (STOMP) to subscribe to `/topic/order/{orderId}` for live updates.
- Driver Client — connects over HTTPS (REST) for status changes, and over WebSocket (STOMP) to publish to `/app/location/{orderId}`.

Application Core (Spring Boot — Waypoint API)

- Controller Layer — accepts REST requests and STOMP message mappings.
- Service Layer — order state machine, nearest-driver matching, ETA computation, authorization checks.
- Repository Layer — JPA repositories over PostgreSQL.

Data & Async Layer

- PostgreSQL (system of record) — Users, Orders, StageHistory, LocationHistory.
- Redis (fast-changing state) — latest driver location per order, rate-limit counters.
- Async Workers (@Async) — history writer, notification dispatch; never block the live broadcast path.

Connection Summary

From	To	Protocol	Purpose
Customer Client	Spring Boot App	HTTPS (REST)	Create/view orders
Customer Client	Spring Boot App	WebSocket (STOMP)	Subscribe to live location updates
Driver Client	Spring Boot App	WebSocket (STOMP)	Publish location updates
Spring Boot App	PostgreSQL	JDBC	Durable reads/writes
Spring Boot App	Redis	Redis protocol	Latest location, rate limits

4.2 Component Responsibilities

Component	Responsibility
Controller Layer	Accepts REST requests (auth, order creation, history queries) and STOMP message mappings (location updates).
Service Layer	Business logic: order state machine, nearest-driver matching, ETA computation, authorization checks.
Repository Layer	JPA repositories for durable entities: User, Order, StageHistory, LocationHistory.
PostgreSQL	System of record. Stores everything that must survive a restart and be queryable historically.
Redis	Ephemeral, fast-changing state: latest known driver location, rate-limit counters, optionally driver availability cache.
WebSocket/STOMP Broker	Manages persistent connections and topic-based message routing between drivers, customers, and the server.
Async Workers (@Async)	Non-blocking side effects: writing location history, dispatching notifications — never on the critical broadcast path.

4.3 Data Flow — End-to-End Trip Walkthrough

- 11.Driver logs in, connects via WebSocket, sets status to ONLINE / AVAILABLE.
- 12.Customer submits an order via REST (POST /orders) with a pickup location.
- 13.Service layer queries available drivers, computes distance to each via the Haversine formula, assigns the nearest one, and transitions the order to ASSIGNED.
- 14.Driver receives the assignment (via WebSocket push to their personal topic) and accepts.
- 15.Driver's client begins sending location updates every few seconds to /app/location/{orderId}.
- 16.On each update: Server (a) writes the latest position to Redis, (b) recomputes ETA, (c) broadcasts position + ETA to /topic/order/{orderId}, and (d) asynchronously appends the point to the durable location history.
- 17.Only the customer who owns that order is subscribed to /topic/order/{orderId}, so only they receive the broadcast.
- 18.Driver updates order status through the stages (PICKED_UP → ON_THE_WAY → DELIVERED), each transition validated and recorded.
- 19.On DELIVERED, the WebSocket topic for that order is closed and the driver's status reverts to AVAILABLE.
- 20.The customer (or a dispatcher) can later retrieve the full historical route for that order from PostgreSQL.

5. Data Model

5.1 Core Entities

Entity	Key Fields	Notes
User	id, name, email, passwordHash, role	role ∈ {CUSTOMER, DRIVER, DISPATCHER}.
DriverProfile	id, userId, status, currentLat, currentLng, lastUpdatedAt	status ∈ {OFFLINE, ONLINE_AVAILABLE, ONLINE_BUSY}.
Order	id, customerId, driverId, pickupLat, pickupLng, dropoffLat, dropoffLng, currentStage, createdAt	The central entity; drives the state machine.
OrderStageHistory	id, orderId, fromStage, toStage, changedAt, actorId	Audit trail of every stage transition.
LocationHistory	id, orderId, lat, lng, recordedAt	Append-only log used for trip replay; written asynchronously.

5.2 Why Location Data Is Split Across Two Stores

This is one of the most important design decisions in Waypoint and a strong interview talking point. The "current location" of a driver changes every few seconds and is only ever needed as its latest value — there is no reason to run a full relational write for every single ping. Redis holds this fast-changing, latest-value-only state. Separately, the full sequence of location points over time has lasting value (route replay, dispute resolution, analytics), so it is durably persisted in PostgreSQL via LocationHistory, written asynchronously so it never sits on the critical real-time path.

5.3 Order State Machine

CREATED → ASSIGNED → PICKED_UP → ON_THE_WAY → DELIVERED

CANCELLED is reachable from CREATED or ASSIGNED only (once a driver has physically picked up the order, cancellation is no longer a clean state transition and must be handled as an operational exception by a dispatcher).

6. API Specification (Summary)

This is a summary specification. Full request/response schemas should be documented as Swagger/OpenAPI annotations directly in code during Week 4, consistent with prior project practice.

6.1 REST Endpoints

Method	Endpoint	Role	Purpose
POST	/auth/register	Public	Register as customer or driver.

Method	Endpoint	Role	Purpose
POST	/auth/login	Public	Authenticate, receive JWT.
POST	/orders	Customer	Create a new order; triggers driver assignment.
GET	/orders/{id}	Customer (owner), Dispatcher	Fetch order details/current stage.
GET	/orders/{id}/history	Customer (owner), Dispatcher	Fetch full location + stage history.
PATCH	/orders/{id}/status	Driver (assigned)	Advance order to next stage.
PATCH	/drivers/me/status	Driver	Toggle online/available/busy.
GET	/dispatch/overview	Dispatcher	Cached operational dashboard.

6.2 WebSocket (STOMP) Channels

Direction	Destination	Purpose
Client → Server	/app/location/{orderId}	Driver publishes a location update.
Server → Client	/topic/order/{orderId}	Broadcast of latest location + ETA to the owning customer.
Server → Client	/topic/driver/{driverId}	Personal channel for order assignment notifications to a driver.

7. Non-Functional Requirements

7.1 Performance

- Location broadcast latency (driver update → customer receives it) should stay under ~1 second under normal load.
- REST endpoints (excluding order creation's matching logic) should respond within 300ms, consistent with the standard set in prior projects.
- The dispatcher overview endpoint must not query the database on every request — it is cached with a short TTL.

7.2 Security & Data Isolation

- A customer must never be able to subscribe to another customer's order topic, even by guessing the order ID — subscription requests are authorized server-side.
- A driver must never be able to submit location updates for an order they are not assigned to.
- Passwords are hashed; JWTs are short-lived with a defined expiry.

7.3 Reliability

- If a driver's WebSocket disconnects mid-order, the system must detect this (not assume they're still active) and flag the order rather than leaving it silently stuck.
- Async history writes must not throw exceptions that affect the main request/broadcast flow (isolated error handling).

7.4 Scalability Considerations (Documented, Not Necessarily Implemented)

Given the one-month/single-instance scope, true horizontal scaling is not implemented, but the design should be able to explain how it would scale, which is exactly what a system design interview probes:

- Horizontally scaling WebSocket servers requires an external message broker (e.g., RabbitMQ or Redis Pub/Sub) instead of Spring's in-memory simple broker, so that a broadcast reaches a subscriber connected to a different server instance.
- Redis remains the shared source of truth for "latest location" across any number of app instances.
- Database writes for history could be batched or moved to a queue (e.g., Kafka) at very high scale instead of direct async writes.

8. Risks & Mitigations

Risk	Impact	Mitigation
WebSocket/STOMP is unfamiliar territory; could consume more time than budgeted.	High	Time-boxed learning in Week 1; fallback to a simplified long-polling approach if truly blocked, documented as a known trade-off.
Authorization on WebSocket subscriptions is easy to get wrong (unlike REST, it's less standardized).	High	Explicit interceptor on the STOMP CONNECT/SUBSCRIBE lifecycle, tested early with two real user accounts, not just one.
Simulating drivers (no real GPS) may feel unconvincing in a demo.	Medium	Build a small script that moves simulated coordinates along a real path between two points, not random jitter — looks convincing on a map.
Scope creep — trying to build a real map UI or real routing.	Medium	Non-Goals section is explicit; stretch features are only touched after MVP is deployed and stable.